

离散数学 2

目录

离散数学 2 1

一、图论基础概念 2026-02-25 2

 (1) 特别基础：图的顶点、边、方向 2

 (2) 握手定理 2

 (3) 赋权图 2

 (4) 特殊的图 2

 完全图：所有点之间互相连接 2

 无向完全图 2

 有向完全图 3

k -正则图 3

 圈图 3

 轮圈 3

 二分图 3

 Ramsey 问题简单记一下 3

 (5) 子图 3

 (6) 同构 3

 同构的判断充要条件 3

 (7) 图的运算 3

二、图的代数结构 2026-03-06 4

 (1) 邻接矩阵 4

 转置 4

 乘积 4

 (2) 关联矩阵 4

 对于无向图， 4

 对于有向图， 4

 (3) 边列表 4

 (4) 正向表 4

 (5) 逆向表 5

 (6) 邻接表 5

 (7) 有向图的十字链表 6

三、道路与回路 2026-03-13 6

 (1) 基本概念 6

 (2) 图的连通性 6

 (3) 割点、割边与连通度 7

 (4) 块 (Block) 8

 (5) 复杂网络简介 8

 (6) 道路与回路的判定方法 8

 (7) 搜索算法 9

四、特殊道路与回路问题 2026-03-17 10

 (1) 欧拉道路与回路 10

 欧拉道路与回路 10

 (2) 哈密顿道路与回路 10

 (3) 旅行商问题与分支定界法 10

 (4) 最短路径 11

 (5) 关键路径 11

 (6) 中国邮路 11

 💧 图论：血泪教训与阴间套路总结 💧 12

 1. 核心铁律：只要边数够多，必有妖孽 12

 2. 情况 B 的必杀技：最长路径论证 (Longest Path Argument) 12

 3. 无三角形 (Triangle-free) 的物理限制 12

 4. 考试/自救心态建设 12

一、图论基础概念 2026-02-25

(1) 特别基础：图的顶点、边、方向

图 G 主要由两部分组成：

1. 顶点集合 V ，里面的东西是节点或者顶点
2. 边集合 E ，是 V 上的二元关系，顶点集合的笛卡尔积上的多重集（即可以有重复的元素）
3. 表示方法

$$V = \{v_1, v_2, v_3, \dots, v_n\}$$

$$E = \{(v_1, v_2), \dots, (v_4, v_5)\}$$

4. 对有限图 $G = (V, E)$

1. 阶 = $|V| = n$
2. $|E| = 0$ 空图
3. $|V| = 1$ 平凡图
4. 未说明是默认有限无向图 一些举例：食物链是有向图

关联同时存在是就是无向图，否则可能是有向图。

5. 无向图不等价于两边都有的有向图

5. 对无向图：1. 认为顶点作为端点时，和它的边有**关联**；普通边关联次数为 1，自环关联次数为 2，不相连关联次数为 0
2. 环、孤立点
3. **顶点相邻**：两个顶点有公共边，和一个点相邻的所有点组成它的邻点集
4. 边相邻：两条边有公共顶点 5. 重边 6. 几种分类：简单图：无重边无自环

多重图：有重边有自环 伪图：都有 7. 度数：所连边的数量，自环贡献两个度；或者就叫**与其相邻边的关联次数之和**

6. 有向图：

1. 通常用 $\langle v_1, v_2 \rangle$
2. 起点/直接前驱**邻接到**终点/直接后继，反之叫做**邻接于**
3. 直接后继集也是**外邻集**
4. 直接前驱集也是**内邻集**
5. 度数：上面**加粗文**：有入度和出度

7. Δ, δ ：最大度最小度 8. 度数列（同样的有入度列、出度列）：

(2) 握手定理

1. 任意图的所有顶点度数之和是边数的 2 倍，有向图的入度=出度=边数
2. !!! 任意图的奇度顶点的个数一定是偶数

有神秘证明，简单理解：度数之和必然为偶数，偶度顶点度数之和必然为偶数，所以奇度顶点度数之和必然为偶数，所以一定有偶数个。

eg.（抽象为图结构的一个例子）不存在具有奇数个面且每个面都具有奇数条棱的多面体。面->点；棱->边

3. 一个非空简单图，一定存在度数相同的顶点

数学归纳法：不存在孤立点、存在一个、存在两个。显然存在两个已经解决了。不存在和存在一个不赖，因为每个点的度数取值范围总比此时的顶点数少 1，所以肯定有相同的。

(3) 赋权图

$w(e)$ 就是 e 的边权。

(4) 特殊的图

完全图：所有点之间互相连接

无向完全图

$$|V_{K_n}| = n(n-1)/2$$

有向完全图

$$m = n(n-1), \Delta^+ = \delta^+ = \delta^- = \Delta^- = n-1, \Delta = \delta = 2(n-1)$$

k -正则图

每个顶点度数均为 k , 边数为 $m = kn/2$

圈图

圈图至少 3 个顶点, 并且是 2-正则图。

轮圈

在圈图里面放一个点, 和圈上的所有点相连。

二分图

两部分只要非空就可以。完全二分图: $K_{m,n}$ 二分图可能有多种剖分方式。

Ramsey 问题简单记一下

本质解决方法就是画出来图, 然后分析实线/虚线多边形的存在可能性。

(5) 子图

1. 定义: $V' \subseteq V, E' \subseteq E$, 此时 $G' \subseteq G$
2. 真子图: $V' \subset V, E' \subset E$
3. 支撑/生成子图: 在定义基础上, $V' = V$
4. 导出子图: 包含某个点集内部所有相连的边集/某个边集关联的所有顶点集。记作 $G[V'], G[E']$ 。

(6) 同构

存在双射函数, 将节点重新编号, 一一对应。 $G_1 \cong G_2$

判断同构的一些必要条件:

- 扭曲变形后和原图一致
- 顶点数、边数相同
- 度数列无序相同
- 对应顶点关联集和邻域相同
- 存在同构的导出子图
- 内部存在一样的封闭图形

eg. 4 阶 3 边无向简单图。

根据握手定理, 总度数为 6, 故最大顶点度数为 3, 奇度点偶数个。现在我们列出来度数列就行了, 然后大概枚举一下。

eg. K_4 的生成子图按照边数划分, 有 1 1 2 3 2 1 1。这对称是因为——补图同构!

同构的判断充要条件

- 对于简单图, 当且仅当补图同构
- 当且仅当邻接矩阵可以通过交换行列得到
- 所有子图都同构

(7) 图的运算

1. 交: $G_1 \cap G_2 = (V_1 \cap V_2, E_1 \cap E_2)$
2. 并: $G_1 \cup G_2 = (V_1 \cup V_2, E_1 \cup E_2)$
3. 差: $G_1 - G_2 = (V_1, E_1 - E_2)$
4. 补: $\overline{G} = (V, E(K_n) - E)$
5. 笛卡尔积: $G_1 \times G_2 = (V_1 \times V_2, E)$, 其中 E 中的边满足: $(u_1, v_1), (u_2, v_2) \in E$ 当且仅当 $u_1 = u_2$ 且 $(v_1, v_2) \in E_2$ 或者 $v_1 = v_2$ 且 $(u_1, u_2) \in E_1$ 。
6. 自补图: G 与 $\overline{G}$ 同构
7. 对称差: $G_1 \oplus G_2 = (V_1 \cup V_2, E_1 \oplus E_2)$
8. $G - v$: 删除顶点 v 和与之关联的边
9. $G - e$: 删除边 e

10. $G + e$: 添加边 e , 如果 e 已经存在则不变
11. $G + v$: 添加顶点 v , 如果 v 已经存在则不变

二、图的代数结构 2026-03-06

(1) 邻接矩阵

对于有向图，每行表示一个顶点的出边，每列表示一个顶点的入边。对于无向图，邻接矩阵是对称的。

对于简单图，邻接矩阵中的元素是 0 或 1，表示是否存在边。对于多重图，元素可以是非负整数，表示边的数量。

对于赋权图，元素可以是边权。此时称为权矩阵，为 0 时不连通。

转置

对于有向图，邻接矩阵的转置对应于将图中的所有边的方向反转。

$$G' = (V, E'), E' = \{(v, w) | (w, v) \in E\}$$

乘积

邻接矩阵的乘积可以用来计算路径数量。例如， A^k 的元素 a_{ij} 表示从顶点 i 到顶点 j 的长度为 k 的路径数量。

(2) 关联矩阵

关联矩阵 M 是一个 n 行 m 列的矩阵，其中 n 是顶点数， m 是边数。

对于无向图，

如果边 e_j 连接顶点 v_i ，则 $M_{ij} = 1$ ；否则为 0。2 表示自环。

性质：

1. 从列来看， $\sum_{i=1}^n b_{ij} = 2, j = 1, 2, \dots, m$
2. 从行来看， $\sum_{j=1}^m b_{ij} = d(v_i), i = 1, 2, \dots, n$
3. 握手定理： $\sum_{i,j} b_{ij} = 2m$
4. 某列某元素为 2： e_j 是自环。
5. 相同的两列： e_j 和 e_k 是重边。
6. 某行全 0： v_i 是孤立点。

对于有向图，

如果边 e_j 从顶点 v_i 出发，则 $M_{ij} = 1$ ；如果边 e_j 到达顶点 v_i ，则 $M_{ij} = -1$ ；否则为 0。

设无环有向图 $D = \langle V, E \rangle$ ，令 $B(D) = (b_{ij})_{n \times m}$ ，满足

$$b_{ij} = \begin{cases} 1, & v_i \text{ 是边 } e_j \text{ 的起点} \\ 0, & v_i \text{ 与边 } e_j \text{ 无关} \\ -1, & v_i \text{ 是边 } e_j \text{ 的终点} \end{cases}$$

矩阵的问题：效率极低，无法利用稀疏矩阵的性质，无法直接反映图的结构特征。

怎么优化？

(3) 边列表

边列表有 2 个 m 维向量 A, B 组成，若 $e_k = (v_i, v_j)$ ，则 $A(k) = i, B(k) = j$ 。

举例：对于图 $G = (V, E)$ ，其中 $V = \{v_1, v_2, v_3\}$ ， $E = \{(v_1, v_2), (v_2, v_3)\}$ ，则边列表为 $A = [1, 2]$ ， $B = [2, 3]$ 。

特点：

- 适合存储稀疏图，节省空间。 $\mathcal{O}(nm) \Rightarrow \mathcal{O}(m)$
- 不适合存储密集图，查询效率低。枚举前驱后继效率低。

(4) 正向表

将节点从 0 到 $n-1$ 编号，将边列表按照起始节点排序，每一个节点的直接后继对应 B 中连续的一段下标区间。 A 的第 $n+1$ 项存储为 m ，表示边的总数。

正向表 (Forward Star) 结构图解

顶点 v	head[v]
1	1
2	3
3	4
4	6
5	7 (哨兵)

下标 i	终点 w	权重
1	v_2	5
2	v_3	2
3	v_4	8
4	v_1	1
5	v_5	3
6	—	—

区间对应（指针指向）：

- 顶点 1 的边区间：edges[1] 到 edges[3-1]
- 顶点 2 的边区间：edges[3] 到 edges[4-1]
- 顶点 3 的边区间：edges[4] 到 edges[6-1]

特点：

1. $d^+(v_i) = A(i+1) - A(i)$
2. $A(i) = \sum_{j=0}^{i-1} d^+(v_j)$
3. 占用空间变成了 $\mathcal{O}(n+m)$ ，适合存储稀疏图。
4. 查询效率高，枚举直接后继效率高。
5. 前驱查询效率低。

(5) 逆向表

与正向表类似，但按照边的终点排序，每一个节点的直接前驱对应 B 中连续的一段下标区间。

无向图的逆向表和正向表是相同的。原因：无向图中边没有方向，正向表和逆向表的定义相同。

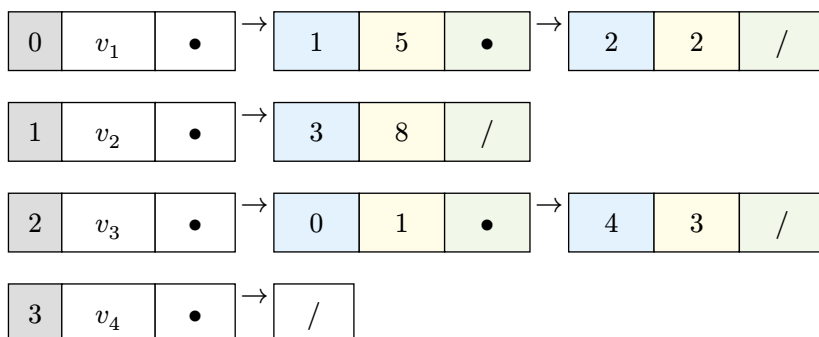
(6) 邻接表

每个节点维护一个链表，链表中存储与该节点相邻的边的信息（如终点和权重）。对于无向图，每条边会在两个节点的链表中出现；对于有向图，每条边只会在起点的链表中出现。起始节点只存储起始顶点。

- dst: 邻接点域
- w: 数据域
- link: 链域

邻接表 (Adjacency List) 存储结构

注：dst-邻接点域 | w-数据域 | link-链域



表头节点结构

data	firstedge
------	-----------

边节点结构

dst	w	link
-----	---	------

典型用途：查询某个节点的直接后继或前驱。复杂度为 $O(d^+(v))$ 或 $O(d^-(v))$ 。

(7) 有向图的十字链表

节点结构如下：

- src: 边的起点
- dst: 边的终点
- weight: 边权
- d_link: 指向同一起点的下一条边
- s_link: 指向同一终点的下一条边

指针非常复杂。

三、道路与回路 2026-03-13

(1) 基本概念

1. **通路 (Path/Walk)**: 给定图 $G = \langle V, E \rangle$, 顶点与边的交替序列 $P = v_0 e_1 v_1 e_2 \dots e_l v_l$ 。
 - 若对所有 i , 边 $e_i = (v_{i-1}, v_i)$, 则称 P 为 v_0 到 v_l 的**通路**。
 - v_0 为**起点**, v_l 为**终点**, l 为通路的**长度**。

表示方法:

1. 按定义用顶点和边的交替序列: $P = v_0 e_1 v_1 e_2 \dots e_l v_l$
 2. 用边序列: $P = e_1 e_2 \dots e_l$
 3. 简单图中, 用顶点序列: $P = v_0 v_1 \dots v_l$
2. **回路 (Circuit)**: 若起点与终点重合 ($v_0 = v_l$), 则称该通路为**回路**, 也称为**圈**。
 - 长度为奇数的圈称为**奇圈**, 长度为偶数的圈称为**偶圈**。
 3. **一些重要事实**:
 - 在无向图中, 长度为 1 的圈由环构成, 长度为 2 的圈由两条重边构成。
 - 在无向简单图中, 所有圈的长度 ≥ 3 。
 - 在有向图中, 长度为 1 的圈由环构成。
 - 在有向简单图中, 所有圈的长度 ≥ 2 。
 4. **分类与特征**:
 - **简单通路/回路**: 通路中所有**边各异**。
 - **初级通路 (路径)**: 简单通路中所有**顶点各异**。
 - **初级回路 (圈)**: 简单回路中除端点外所有**顶点各异**。
 - **关系**: 初级通路/回路必是简单通路/回路, 但反之不一定成立。
 - **例子**:
 1. 初级通路: $v_1 v_2 v_3 v_4$ (所有顶点不同)
 2. 非初级的简单通路: $v_1 v_2 v_3 v_2 v_4$ (边各异但顶点重复)
 3. 初级回路: $v_1 v_2 v_3 v_1$ (除起点终点外顶点不同)
 4. 非初级的简单回路: $v_1 v_2 v_3 v_2 v_1$ (边各异但内部顶点重复)
 5. **圈的长度限制**:
 - 无向简单图中, 所有圈的长度 ≥ 3 。
 - 有向简单图中, 所有圈的长度 ≥ 2 。
 6. **存在性定理**: 在无向图 G 中, 若任意顶点 v 的度数 $d(v) \geq 2$, 则 G 中一定存在圈。

证明: 不妨设 G 是简单图。如果 G 中存在圈, 得证。否则 G 中不存在圈, 则必然可在 G 中找到一条最长的路径 $L: v_0 v_1 \dots v_l$ 。考虑到 v_0 至少存在 2 个邻居, v_1 是其中之一, 假设 v_x 是另外一个邻居。

- 如果 v_x 在 L 中出现, 则 $v_x v_0 v_1 \dots v_x$ 成圈, 矛盾;
- 否则 $v_x v_0 v_1 \dots v_l$ 是一条更长的路径, 矛盾。

因此 G 中一定存在圈。

(2) 图的连通性

1. 无向图的连通性:

- **连通 (Connected)**: 顶点 u 与 v 之间存在通路。
- **连通图**: 任意两点都连通的图。

- **连通支 (Connected Component):** 图的极大连通子图。 $p(G)$ 表示连通分支数。
 - **判定:** 若简单图的边数 $m > \frac{1}{2}(n-1)(n-2)$, 则 G 是连通图。
2. 有向图的连通性:
- **可达 (Reachable):** u 到 v 有通路。
 - **强连通:** 任意两点 u, v 相互可达。
 - **单向连通:** 对于任意两点 u, v , u 可达 v 或 v 可达 u 。
 - **弱连通:** 略去方向后的无向图是连通图。
 - **强连通 $\rightarrow$ 单向连通 $\rightarrow$ 弱连通** (反之不真)。
3. 短程线与距离:
- **短程线:** 两点间长度最短的通路。
 - **距离 $d(u, v)$:** 短程线的长度。不连通时规定为 ∞ 。
 - 有向图距离不满足对称性 (即 $d < u, v >$ 不一定等于 $d < v, u >$)。
- (3) 割点、割边与连通度
1. **割点 (Cut-vertex):** 若 $G - v$ 的连通分支数比 G 多, 则称 v 为割点。 **等价性质:**
- v 是割点当且仅当存在 u, w 使得所有 u 到 w 的路径都经过 v 。
 - v 是割点当且仅当图 $V - v$ 可以划分为两个顶点集 U 和 W , 使得对任意顶点 $u \in U$ 和 $w \in W$, 顶点 v 都在每条道路 $P_{\{uw\}}$ 上。
2. **割边 (Cut-edge/Bridge):** 若 $G - e$ 的连通分支数比 G 多, 则称 e 为割边。 **等价性质:**
- e 是割边当且仅当 e 不属于任何回路。
 - e 是割边当且仅当存在 G 的顶点 u 和 w , 使 e 属于 u 和 w 的任何一条道路 $P_{\{uw\}}$ 。
 - e 是割边当且仅当图 $G - e$ 可以划分为两个顶点集 U 和 W , 使得对任意顶点 $u \in U$ 和 $w \in W$, 道路 $P_{\{uw\}}$ 都经过 e 。
3. **点连通度 $\kappa(G)$:** 使图不连通或只剩一个孤立点所需删除的最小顶点数。
- 若有割点, 则 $\kappa(G) = 1$; 不连通图为 0; K_n 为 $n - 1$ 。
4. **边连通度 $\lambda(G)$:** 使图不连通所需删除的最小边数。
5. **重要定理:** 对任意连通图 G , 有 $\kappa(G) \leq \lambda(G) \leq \delta(G)$ (其中 $\delta(G)$ 为最小度)。

证明:

第一步: 证明 $\lambda(G) \leq \delta(G)$

- 存在点 v 的度数是 $\delta(G)$
- 删掉与 v 相邻的所有边 (共 $\delta(G)$ 条) 可以使得 v 与其他节点不连通
- 因此 $\lambda(G) \leq \delta(G)$

第二步: 证明 $\kappa(G) \leq \lambda(G)$

- 设 $B = \{e_1, e_2, \dots, e_\lambda\}$ 为 G 的一最小边断集, $G - B$ 含两个不连通顶点子集 V_1, V_2
- 删除每条边在 V_1 中的端点, 则 B 中的边不复存在, 导致 V_1 和 V_2 不连通
- 极端情况下, V_1 被删空, 剩下的 V_2 不一定不连通
- **解决方法:** 留一个
 1. 删掉 $e_1, e_2, \dots, e_{\lambda-1}$ 在 V_1 中的端点
 2. 删掉 e_λ 在 V_2 中的端点
 3. 这样至多删掉 λ 个顶点使图不连通

• **更严谨的证明:**

1. 如果 G 有一个顶点 v 不在 B 中的任何边上, 设 C 是 $G - B$ 后包含 v 的那个连通支, 删掉 C 中的 B 的端点后 G 变得不连通, $\kappa(G) \leq \lambda(G)$
2. 如果不存在这样的 v , 即图中所有节点都关联 B 中的边, 此时假定 v 是其中任意一个节点, 那么 v 所有邻接节点必然都对应了 B 中不同的边, 因此 v 的邻接点数量不超过 λ , 删掉这些点将 v 和 $G - v$ 分开, 得证

- **推论:** 对于简单连通图 G , 有 $\kappa(G) \leq 2\frac{m}{n}$, 其中 m 为边数, n 为顶点数。

- **证明：**由握手定理， $\sum d(v_i) = 2m$ ，故 $n\delta(G) \leq 2m$ ，即 $\delta(G) \leq 2\frac{m}{n}$ 。再由前述定理得 $\kappa(G) \leq \delta(G) \leq 2\frac{m}{n}$ 。

6. 点割集与边割集：

- **点割集：**若删除顶点集合 V' 后图的连通分支数增加，且删除 V' 的任何真子集都不会增加连通分支数，则称 V' 为点割集。
- **边割集：**若删除边集合 E' 后图的连通分支数增加，且删除 E' 的任何真子集都不会增加连通分支数，则称 E' 为边割集。
- **点断集：**使连通图变得不连通或只剩一个孤立节点的顶点集合。
- **边断集：**使连通图变得不连通的边集合。

(4) 块 (Block)

- **定义：**图中极大的没有割点的连通子图。

定理 2.1.3：设 G 是至少有 3 个顶点的连通图，则下述性质等价：

1. G 是一个块。
2. G 的任何两个顶点同属某一初级回路。
3. G 的任何一个顶点和任何一条边同属某个初级回路。
4. G 的任何两条边同属某个初级回路。
5. 给定两个点 u, v 和一条边 e ，存在一条包含 e 的初级道路 P_{uv} 。
6. 对 G 的任意三个不同的顶点，存在一条包含它们的初级道路。
7. 对 G 的任意三个不同的顶点，存在一条只包含其中两点而不含第三点的道路。

(5) 复杂网络简介

- **小世界现象：**大部分节点彼此不相连，但绝大部分节点之间经过少数几步就可到达。
 1. 特征：高集聚系数和低平均路径长度
 2. 六度分隔理论：任意两人之间“距离”约为 6
 3. 贝肯数、埃尔德什数
- **无尺度特征：**大部分节点只和很少节点连接，极少数节点与非常多节点连接。
 1. 度分布：幂律分布 $P(d = k) \sim k^{-\gamma}$ ， γ 通常在 2 到 3 之间。

(6) 道路与回路的判定方法

1. **代数方法——邻接矩阵幂：**
 - 设 A 是图 G 的邻接矩阵，则 A^l 中 (i, j) 位置元素为从顶点 v_i 到 v_j 长度为 l 的路的数目。
 - 证明：对 l 归纳， $A^l = A^{l-1} * A$ 。
2. **可达矩阵：**
 - $P(G) = A + A^2 + \dots + A^n$ 或 $P = A \vee A^2 \vee \dots \vee A^n$ （逻辑运算）
 - 若只关心可达性，可用逻辑运算法，复杂度 $O(n^4)$ 。
3. **Warshall 算法：**

```
P = [a_(i j)]_(n times n)
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      p_(i j) = p_(i j) or (p_(i k) and p_(k j))
```

- 复杂度： $O(n^3)$
 - 原理： $p_{ij}^{(k)} = p_{ij}^{(k-1)} \vee (p_{ik}^{(k-1)} \wedge p_{kj}^{(k-1)})$
 - $p_{ij}^{(k)} = 1$ 当且仅当顶点集 $\{v_i, v_1, v_2, \dots, v_k, v_j\}$ 之间有 v_i 到 v_j 的道路。
4. **所有节点对最短路 (Floyd 算法)：**

```
P = [a_(i j)]_(n times n) // 邻接=1, 不邻接=$infinity$
for k = 1 to n:
  for i = 1 to n:
```

```
for j = 1 to n:
    P_(i j) = min(P_(i j), P_(i k) + P_(k j))
```

5. 应用举例——序列整除问题:

- 问题: 任意给定 $(n+1)^2$ 项的递增自然数列, 证明以下结论必有一条成立: (1) 存在 $n+1$ 项的子列, 使任一项能整除此子列中它后面的每一项; (2) 存在 $n+1$ 项的子列, 使此子列中任一项不能整除此子列中它后面的任何一项。
- 解法: 建立有向图模型, 顶点为数列元素, 若 a_i 整除 a_j 则有边 (a_i, a_j) 。
- 对每个顶点 v_i , 定义 $l(v_i)$ 为以 v_i 为起点的最长有向路径长度。
- 若存在 $l(v_i) = n$, 则结论(1)成立。
- 否则所有 $l(v_i) \leq n-1$, 由鸽巢原理, 存在 $n+1$ 个顶点具有相同的 $l(v_i)$ 值, 这些顶点互不相邻, 结论(2)成立。

(7) 搜索算法

1. 搜索基本概念:

- 基本依据: 邻接关系
- 原则: 每个节点只访问 1 次, 迭代到没有新的可访问节点为止。
- 节点状态:
 1. Invisible: 还未被发现的节点
 2. Visible: 已被发现 (未访问, 访问过前驱节点)
 3. Visited: 已访问过
- 通用搜索框架:

```
visible = ${v_0}$; // $v_0$ 是起始节点
visited = {}
while |visible| > 0:
    v = fetch(visible); // 从可见节点中取出一个
    visit(v); // 访问该节点
    visited = visited union ${v}$;
    visible = (visible union ${\Gamma^+(v)}$) setminus visited;
```

- **fetch** 策略决定搜索方式。

2. 宽度优先搜索 (BFS):

- 策略: 自上往下, 逐层选择; 越靠上 (先发现) 越优先。
- 数据结构: **队列** (先发现, 先访问)。
- 特点: 找到的是**最短路** (步数最少)。

```
BFS(v):
    flag[v] = 1; // 1=visible, 2=visited, 0=invisible
    Q = [v]; // 队列初始化
    while Q 非空:
        u = DEQUEUE(Q);
        for 邻接于 u 的所有顶点 w:
            if flag[w] = 0:
                ENQUEUE(w, Q);
                flag[w] = 1;
                pred[w] = u; // 记录前驱以还原路径
        flag[u] = 2;
```

3. 深度优先搜索 (DFS):

- 策略: 优先选择新发现的节点扩展; 越靠下 (晚发现) 越优先。
- 数据结构: **栈** (后发生, 先访问)。
- 特点: 尽量向下搜索, 只有碰壁才回头。

```

DFS(v):
    visited[v] = 1;
    for 邻接于 v 的每个顶点 w:
        if visited[w] = 0:
            DFS(w); // 递归

```

4. 应用举例——过河问题：

- 人带狼、羊、白菜过河，每次只能带一样，狼羊、羊菜不能独处。
- 解法：建立图模型，16 种状态中排除非法状态剩 10 种合法状态，求从初始到目标的路径。

四、特殊道路与回路问题 2026-03-17

(1) 欧拉道路与回路

欧拉道路与回路

欧拉道路：经过图中每条边恰好一次的道路。

欧拉回路：经过图中每条边恰好一次的回路。

欧拉图：包含欧拉回路的图。

半欧拉图：包含欧拉道路但不包含欧拉回路的图。

判定定理：

- 无向图 G 存在欧拉回路当且仅当 G 连通且所有顶点度数均为偶数。
- 无向图 G 存在欧拉道路（但非回路）当且仅当 G 连通且恰有两个奇度顶点。
- 有向图 D 存在欧拉回路当且仅当 D 强连通且每个顶点入度等于出度。
- 有向图 D 存在欧拉道路（但非回路）当且仅当 D 弱连通，且除两个顶点外其余顶点入度等于出度，其中一个顶点出度比入度大 1（起点），另一个顶点入度比出度大 1（终点）。
-

Fleury 算法（构造欧拉回路）：

1. 任选一个顶点作为起点
2. 在当前顶点的邻接边中，优先选择非桥边（删除后不增加连通分支数的边）
3. 删除已走过的边，移动到下一个顶点
4. 重复直到所有边都被遍历

(2) 哈密顿道路与回路

<哈密顿道路与回路 >

哈密顿道路：经过图中每个顶点恰好一次的道路。 **哈密顿回路：**经过图中每个顶点恰好一次的回路。

重要性质：

- 哈密顿图：包含哈密顿回路的图
- 判定哈密顿图是 NP 完全问题，无有效的一般判定算法

充分条件：

- **Dirac 定理：**对于 $n \geq 3$ 的简单图 G ，若每个顶点度数 $\geq n/2$ ，则 G 是哈密顿图。
- **Ore 定理：**对于 $n \geq 3$ 的简单图 G ，若任意两个不相邻顶点 u, v 满足 $d(u) + d(v) \geq n$ ，则 G 是哈密顿图。

必要条件：

- 若 G 是哈密顿图，则对 V 的任意非空真子集 S ，有 $p(G-S) \leq |S|$ ，其中 $p(G-S)$ 是 $G-S$ 的连通分支数。

(3) 旅行商问题与分支定界法

<旅行商问题与分支定界法 >

旅行商问题（TSP）：给定 n 个城市和它们之间的距离，求访问每个城市恰好一次并回到起点的最短回路。

分支定界法：

- 基本思想：系统地枚举所有可能解，但在搜索过程中剪枝不可能产生最优解的分支

- 步骤：
 1. 计算初始下界（如最小生成树权重的 2 倍）
 2. 选择一个城市作为根节点开始分支
 3. 对每个分支计算下界，如果下界大于当前最优解，则剪枝
 4. 继续分支直到找到最优解或所有分支都被剪枝

(4) 最短路径

<最短路径 >

Dijkstra 算法（单源最短路径，非负权）：

```
Dijkstra(s):
  for each vertex v:
    dist[v] = infinity
    prev[v] = undefined
  dist[s] = 0
  Q = all vertices
  while Q is not empty:
    u = vertex in Q with min dist[u]
    remove u from Q
    for each neighbor v of u:
      alt = dist[u] + weight(u, v)
      if alt < dist[v]:
        dist[v] = alt
        prev[v] = u
```

Floyd-Warshall 算法（所有点对最短路径）：

```
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

(5) 关键路径

<关键路径 >

在 AOE 网（Activity On Edge network）中：

- **关键路径**：从源点到汇点的最长路径
- **关键活动**：关键路径上的活动
- **最早开始时间**： $\text{earliest}[v] = \max\{\text{earliest}[u] + \text{weight}(u,v)\}$
- **最晚开始时间**： $\text{latest}[v] = \min\{\text{latest}[w] - \text{weight}(v,w)\}$
- **松弛时间**： $\text{slack} = \text{latest} - \text{earliest}$

关键路径决定了整个工程的最短完成时间。

(6) 中国邮路

<中国邮路 >

中国邮路问题：邮递员需要走过每条街道至少一次并返回起点，求最短路线。

解法：

- 如果图中所有顶点度数为偶数，则存在欧拉回路，直接使用欧拉回路即可
- 如果存在奇度顶点（必为偶数个），需要添加重复边使所有顶点度数变为偶数
- 最优解：在奇度顶点之间找最小权匹配，添加对应路径上的边

算法步骤：

1. 找出所有奇度顶点
2. 计算奇度顶点之间的最短路径距离
3. 求奇度顶点的最小权完美匹配
4. 在匹配的顶点对之间添加最短路径上的边

5. 在新图上求欧拉回路

💧 图论：血泪教训与阴间套路总结 💧

记录于：2026-03-18 | 耗时：5 小时的心态爆炸

1. 核心铁律：只要边数够多，必有妖孽

在简单图中， n 个点如果想要“安分守己”（连通且无环），最多只能有 $n - 1$ 条边。

- **定理：** 若 $m \geq n$ ，则 **图中必有回路 (Cycle)**。
- **技巧：** 证明此结论最快的两把刀：
 1. **数学归纳法：** 找到度数 $d(v) \leq 1$ 的废点，删掉它，把锅甩给 $n - 1$ 的情况。
 2. **最长路径法：** 在全员度数 $d \geq 2$ 时，拉出一条最长路径，终点因为没处可去，只能**往回连老熟人**，瞬间闭环。

2. 情况 B 的必杀技：最长路径论证 (Longest Path Argument)

这是今天卡了最久的坑。当图里所有点都“很顶”（度数 $d \geq 3$ 或 $d \geq 2$ ）时，不要空想，直接祭出此招：

- **严谨步骤：**
 1. 设 $P = (v_1, v_2, \dots, v_k)$ 为图中最长路径。
 2. 终点 v_k 的所有邻居 $N(v_k)$ **必须全部在路径 P 上**（否则路径能更长，矛盾！）。
 3. 如果 $d(v_k) \geq 2$ ，它至少连了一个之前的点 v_i ，形成 **回路**。
 4. 如果 $d(v_k) \geq 3$ ，它至少连了两个之前的点 v_i, v_j ，在大圈上又拉了一根捷径，形成 **带弦回路 (Chorded Cycle)**。

3. 无三角形 (Triangle-free) 的物理限制

看到“无三角形”，脑子里第一反应不是组合数，而是 **朋友圈互斥**。

- **核心推导：**
 - G 无三角形 $\implies$ 任何相邻两点 u, v ，其邻居集合 **交集为空**。
 - 导出重要不等式： **$d(u) + d(v) \leq n$** 。
 - 所有的平方和证明（ $\sum d^2$ ）和边数上限证明（ $m \leq \frac{n^2}{4}$ ），全部是从这一个 **物理隔绝** 的点出发的。

4. 考试/自救心态建设

- **教训：** 图论证明题不是靠逻辑硬掰的，是靠 **辅助结构**（最长路径、极大图、二部图）切入的。
- **反思：** 以后死磕超过 30 分钟没思路，立刻检查是否漏掉了以下套路：
 - **删点减规模**（归纳法）
 - **最长路径找端点**（构造回路）
 - **二部图缩放**（极值边界）

“五个小时不会白费，这些套路现在是我的了。”